03_SECURITY_ARCHITECTURE

SUOP ERP v1.0 — Enterprise Security Architecture

Field	Value
Document Version	1.0

Field	Value
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner
Compliance Frameworks	OWASP Top 10, ISO 27001, SOC 2, FSSAI Cybersecurity

---

1. Purpose

This document defines the complete security architecture for SUOP ERP v1.0. It is informed by OWASP Top 10 (2021), ISO 27001 controls, SOC 2 Trust Services Criteria, and FSSAI cybersecurity guidelines for food businesses.

Hard rule: No code ships without passing security review against this document. CI runs SAST (Static Application Security Testing), dependency scanning, and secret detection on every PR.

---

2. Threat Model

2.1 Assets to Protect

Asset	Sensitivity	Impact of Breach
User credentials (passwords, tokens)	Critical	Account takeover, data exfiltration
PII (employee, customer data)	High	Legal, regulatory, reputational
Financial data (prices, costs, invoices)	High	Competitive harm, fraud

Asset	Sensitivity	Impact of Breach
Recipe/formula data	Critical	IP theft, competitive loss
Quality data (NCRs, COAs, audit logs)	High	Regulatory non-compliance
Batch genealogy / recall data	Critical	Food safety liability
Supplier contracts / pricing	High	Competitive harm
Audit logs	High	Loss of legal evidence

2.2 Threat Actors

Actor	Capability	Motivation
External attacker	Internet-facing exploit, phishing	Financial, ransomware
Malicious insider	Authenticated access, knows system	Sabotage, data theft
Negligent insider	Authenticated access	Accidental disclosure
Compromised supplier	Limited tenant access	Pivot attack
Script kiddie	Automated scans	Notoriety

2.3 Attack Surfaces

1. **Public API** — `/api/v1/*` (auth required except login)
2. **Frontend** — Next.js app (XSS, CSRF)
3. **Database** — Direct (should be impossible from internet)
4. **File storage** — S3/MinIO (signed URLs, bucket policy)
5. **Admin endpoints** — `/api/v1/_internal/*` (API key required)
6. **CI/CD pipeline** — Compromise deploys malicious code

7. **Developer machines** — Source code, secrets
8. **Third-party integrations** — Email, SMS, payment providers

---

3. Authentication

3.1 Password Policy

Rule	Requirement
Minimum length	12 characters
Maximum length	128 characters (prevents DoS via long hash)
Complexity	At least 3 of: uppercase, lowercase, digit, special
Common password check	Reject top 10,000 common passwords (HaveIBeenPwned API + local list)
History	Cannot reuse last 10 passwords
Expiry	90 days (configurable per tenant; disabled if MFA enabled)
Lockout	5 failed attempts → 30-min lockout; 3 lockouts in 24h → admin unlock required
Reset	Token-based, single-use, expires in 30 min

3.2 Password Storage

- **Algorithm:** Argon2id (winner of Password Hashing Competition)
- **Parameters:** `memoryCost=19456` (19 MB), `timeCost=2`, `parallelism=1`

- **Tuning target:** ~250ms per hash on production hardware
- **Salt:** 16-byte random salt per password (Argon2 handles internally)
- **Format:** `$argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash>`

Forbidden: MD5, SHA-1, SHA-256 (without salt+pepper), bcrypt with cost < 12, PBKDF2 with < 100k iterations.

3.3 Password Pepper

- Argon2id hash further encrypted with a server-side pepper (32-byte random)
- Pepper stored in secrets manager (not in DB, not in code)
- Compromise of DB alone does not reveal passwords
- Pepper rotation requires password reset for all users (rare; documented procedure)

3.4 Login Flow

1. Client sends `POST /api/v1/auth/login` with `{email, password}`
2. Server loads user by email (rate-limited: 10/min/IP, 5/min/email)
3. Server verifies password with Argon2id (`argon2.verify`)
4. If MFA enabled: return `{ status: 'MFA_REQUIRED', mfaSessionToken }` (5-min TTL)
5. Client sends `POST /api/v1/auth/mfa/verify` with `{mfaSessionToken, code}`
6. Server verifies TOTP, issues access token (15 min) + refresh token cookie (30 days)
7. Login audit logged; suspicious login (new IP, new device) triggers email alert

3.5 Failed Login Protection

- 5 failed attempts per user → 30-min lockout
- 10 failed attempts per IP → 1-hour IP block
- Failed attempts logged; 10+ failures in 1 hour trigger security alert
- Account lockout requires password reset (forces user to verify identity via email)

3.6 Session Management

- **Access token:** JWT, 15 min TTL, in-memory in frontend (not localStorage — XSS protection)
- **Refresh token:** Random 64-byte token, 30-day TTL, httpOnly secure cookie
- **Refresh token rotation:** Every refresh issues a new refresh token; old one invalidated
- **Refresh token replay detection:** Reuse of invalidated refresh token revokes entire session
- **Device tracking:** Each session tied to a device fingerprint; new device triggers email alert

3.7 Multi-Factor Authentication (MFA)

- **TOTP-based** (RFC 6238) — Google Authenticator, Authy, 1Password
- **Required for:**
  - All admin roles (`tenant:admin`, `system:*`)
  - Finance roles (`finance:*`)
  - Quality roles with `ncr:approve`, `coa:sign`, `recall:initiate`
- **Optional for:** Other users (configurable per tenant)
- **Backup codes:** 10 single-use codes generated at enrollment; hashed in DB
- **Enrollment:** User scans QR code → enters code to confirm → backup codes displayed once
- **Reset:** Admin can reset MFA (audited at `CRITICAL`); user must re-enroll

3.8 Single Sign-On (Future)

- SAML 2.0 (Microsoft Entra, Google Workspace) — Phase X
- OIDC (OAuth 2.0 extension) — Phase X
- Out of scope for Phase 0; architecture allows addition without rewrite

---

4. Authorization

4.1 RBAC Model

```
User → Role → Permission → Resource:Action
```

- **User:** authenticated identity
- **Role:** collection of permissions (e.g., `quality_manager`, `procurement_officer`)
- **Permission:** specific action on resource (`po:approve`, `product:create`)
- **Resource:** entity type (`purchase_order`, `product`, `user`)

4.2 Permission Catalog

All permissions catalogued in `packages/shared/src/enums/permission.enum.ts`. Examples:

```ts
export enum Permission {
  // Organization
  ORG_READ = 'org:read',
  ORG_CREATE = 'org:create',
  ORG_UPDATE = 'org:update',
  ORG_DELETE = 'org:delete',
  // Auth
  AUTH_MANAGE_USERS = 'auth:manage_users',
  AUTH_MANAGE_ROLES = 'auth:manage_roles',
  AUTH_RESET_PASSWORD = 'auth:reset_password',
  // Product
  PRODUCT_READ = 'product:read',
  PRODUCT_CREATE = 'product:create',
  PRODUCT_UPDATE = 'product:update',
  PRODUCT_DELETE = 'product:delete',
  // Purchase Order
  PO_READ = 'po:read',
  PO_CREATE = 'po:create',
```

```
  PO_APPROVE = 'po:approve',
  PO_APPROVE_ANY =
        'po:approve:any',  // approve any PO, not just
        assigned
  PO_CANCEL = 'po:cancel',
  // Quality
  NCR_CREATE = 'ncr:create',
  NCR_APPROVE = 'ncr:approve',
  COA_SIGN = 'coa:sign',
  RECALL_INITIATE = 'recall:initiate',
  // System
  SYSTEM_TENANT_CROSS = 'system:tenant:cross',
  SYSTEM_REFERENCE_UPDATE = 'system:reference:update',
  AUDIT_READ = 'audit:read',
  AUDIT_READ_CRITICAL = 'audit:read:critical',
}
```

4.3 Permission Enforcement (3 layers — see API Standards §13)

1. **Route-level middleware:** `requirePermission('po:create')` blocks unauthorized users before controller
2. **Service-level row checks:** Verify user can access specific record (e.g., PO assigned to them)
3. **Field-level response shaping:** Strip fields user lacks permission to see (e.g., `costPrice`)

4.4 Default Roles

Role	Permissions
`tenant_admin`	All `*:read/create/update/delete` within tenant
`quality_manager`	`quality:*`, `ncr:*`, `capa:*`, `coa:*`, `audit:read`
`quality_inspector`	

Role	Permissions
	`quality:read`, `iqc:inspect`, `ncr:create`, `coa:read`
`procurement_officer`	`po:read/create`, `supplier:read`, `grn:read`
`procurement_manager`	`po:*`, `supplier:*`, `grn:create`
`warehouse_operator`	`inventory:read`, `grn:post`, `putaway:execute`
`warehouse_manager`	`warehouse:*`, `inventory:*`, `grn:*`
`production_manager`	`production:*`, `recipe:read`, `bom:read`
`plant_head`	All within plant scope
`finance_manager`	`finance:*`, `cost:*`, `invoice:*`
`auditor`	`*:read`, `audit:read:critical` (read-only)
`system_admin`	`system:*` (platform team only)

4.5 Permission Inheritance

- Roles are flat (no inheritance) — simpler, more predictable
- Users can have multiple roles; permissions union
- Tenant admin role granted via separate UI (audited at `CRITICAL`)

4.6 Permission Changes Audit

- Every role assignment / revocation audit-logged
- Every permission change audit-logged
- High-severity changes (`system:*`, `tenant_admin` grant) trigger security alert

---

5. JWT Strategy

5.1 Token Structure (see API Standards §12.2)

5.2 Signing Key Management

- **Phase 0:** Single HMAC-SHA256 key in secrets manager
- **Key rotation:** Every 90 days; old key valid for 7 days after rotation (overlap)
- **Multi-key support:** JWT header includes `kid` (key ID) to identify signing key
- **Future (RS256):** Private key in HSM/KMS; public key distributed to verifiers

5.3 Token Validation

On every request: 1. Extract `Authorization: Bearer <token>` header 2. Verify signature (correct key, correct algorithm — reject `alg: none` attacks) 3. Verify `exp` not expired 4. Verify `iss` matches `suop-erp` 5. Verify `aud` matches `suop-frontend` (or `suop-mobile` for mobile) 6. Check `jti` not in Redis blocklist 7. Load user from DB (cached 60s) — verify `status === 'ACTIVE''` 8. Set `RequestContext.user` and `RequestContext.tenant`

5.4 Token Compromise Response

If a token is compromised: 1. Add `jti` to Redis blocklist (TTL = remaining token TTL) 2. Revoke all user's refresh tokens (force re-login) 3. Audit log at `CRITICAL` severity 4. Notify user via email + in-app 5. Optional: Force password reset

---

6. Refresh Token Strategy

6.1 Storage

- **Client:** httpOnly secure cookie (`refresh_token=<random>`; SameSite=Strict; Secure; HttpOnly)
- **Server:** Hashed (SHA-256) in `refresh_tokens` table with `userId`, `expiresAt`, `deviceFingerprint`, `revokedAt`

6.2 Refresh Flow

1. Access token expired → client receives `401`
2. Client calls `POST /api/v1/auth/refresh` (no body; refresh cookie sent automatically)
3. Server validates cookie token against DB:
    - Exists in DB? (not revoked)
    - Not expired?
    - Device fingerprint matches?
4. Issue new access token + new refresh token
5. **Old refresh token immediately revoked** (replay detection)
6. New refresh token in cookie

6.3 Refresh Token Replay Detection

If a revoked refresh token is used: 1. Server detects reuse (token in DB but `revokedAt` set) 2. **Revoke entire session** (all refresh tokens for this user) 3. Force re-login 4. Audit log at `CRITICAL` severity 5. Alert security team

This protects against refresh token theft — attacker using stolen token alerts legitimate user.

6.4 Refresh Token Revocation

- **User logout:** `POST /api/v1/auth/logout` revokes current refresh token
- **Admin force-logout:** `POST /api/v1/_internal/users/{id}/revoke-sessions` revokes all sessions

- **Password change:** All refresh tokens revoked (force re-login everywhere)
- **Account lockout:** All refresh tokens revoked

---

7. Password Policy

(See §3.1 for full policy)

7.1 Password Strength Estimation

In addition to character rules, use `zxcvbn` to estimate password strength: - Score 0-2: Reject (too weak) - Score 3: Acceptable (warning shown) - Score 4: Strong - Score 5: Excellent

7.2 Password Reset Flow

1. User clicks “Forgot password” → enters email
2. Server generates single-use reset token (UUID v7, 30-min TTL)
3. Token stored hashed in `password_reset_tokens` table
4. Email sent with link `https://app.sudhamrit.com/reset-password?token=...`
5. User clicks link, enters new password
6. Server validates token (exists, not expired, not used)
7. Server updates password (Argon2id hash), invalidates token
8. **All existing sessions revoked** (force re-login everywhere)
9. Audit log entry; email notification to user

7.3 Password Reset Protection

- Rate limited: 3 reset requests per email per hour (prevents email bombing)
- Token single-use (reuse → rejected)
- Token expires 30 min after issue
- Reset invalidates all sessions (prevents attacker maintaining access after legitimate reset)

---

8. MFA (see §3.7)

---

9. Encryption

9.1 Encryption Layers

Layer	Algorithm	Purpose
Transport (HTTPS)	TLS 1.3	Data in transit
Database connection	TLS 1.3	App ↔ DB
Database at rest	AES-256 (Supabase/ RDS managed)	DB files on disk
File storage at rest	AES-256 (S3 SSE-KMS)	S3 objects
Backup encryption	AES-256 (pg_dump `--encrypt` or S3 SSE)	Backups
Secrets at rest	KMS-managed	Secrets in secrets manager
Sensitive column encryption	pgcrypto (AES-256-GCM)	PII columns
JWT signing	HS256 (HMAC-SHA256)	Token integrity
Password hashing	Argon2id	Password storage
TOTP secret storage	AES-256-GCM (pgcrypto)	MFA secrets

9.2 TLS Configuration

- **Minimum version:** TLS 1.3 (TLS 1.2 only for legacy clients; disabled by 2027)

- **Cipher suites:** TLS_AES_256_GCM_SHA384, TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256
- **HSTS:** `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`
- **Certificate:** Let's Encrypt (auto-renewed) or managed cert (Supabase/AWS)
- **Certificate pinning:** Not used (CA-based trust sufficient; pinning causes client breakage on rotation)

9.3 Sensitive Column Encryption

For PII / regulated data:

```
-- Aadhaar number stored encrypted
ALTER TABLE users ADD COLUMN aadhaar_encrypted BYTEA;
-- Encrypt on insert
INSERT INTO users (..., aadhaar_encrypted) VALUES (...,
        pgp_sym_encrypt($aadhaar, $key));
-- Decrypt on read
SELECT pgp_sym_decrypt(aadhaar_encrypted, $key) AS aadhaar
        FROM users WHERE id = $1;
```

- Encryption key in secrets manager (not in DB)
- Key rotation requires re-encrypting all rows (documented procedure; tested in DR drill)

9.4 Application-Level Encryption

For especially sensitive data (recipe formulas, supplier contracts): - Encrypt in application code before storing - Decryption key in secrets manager - Key access audited (every decryption logged)

---

10. Secrets Management

10.1 Secret Hierarchy

Type	Storage	Examples
Application secrets	`.env` (dev) / secrets manager (prod)	DB URL, JWT secret, API keys
Infrastructure secrets	Secrets manager only	DB password, S3 access keys
User secrets	DB (hashed)	Passwords, MFA secrets
Certificates	Secrets manager + filesystem	TLS certs, signing keys

10.2 Secrets Manager

- **Production:** AWS Secrets Manager or HashiCorp Vault
- **Staging:** Same as production (separate instance)
- **Development:** `.env` file (gitignored, .env.example committed)
- **CI/CD:** GitHub Actions secrets

10.3 Secret Rotation

Secret	Rotation Frequency	Method
JWT signing key	90 days	Multi-key overlap (old valid 7 days)
DB password	90 days	Rotate via secrets manager; app reconnects
S3 access keys	90 days	IAM role preferred (no key rotation)

Secret	Rotation Frequency	Method
SMTP password	180 days	Manual
Encryption keys	365 days	Re-encrypt all data (tested in DR drill)
API keys (integrations)	365 days	Issue new + revoke old

10.4 Secret Access Audit

- Every secret read from secrets manager logged
- Unusual access patterns alert security (e.g., secret read from new IP)
- Production secret access requires MFA (for humans)

10.5 Forbidden

- ❌ Secrets in code (even commented out)
- ❌ Secrets in git (commit hooks block via `git-secrets`)
- ❌ Secrets in CI logs (auto-redacted)
- ❌ Secrets in error messages
- ❌ Secrets in URLs (URLs logged everywhere)

---

11. OWASP Top 10 Coverage

11.1 A01: Broken Access Control

- **Mitigation:** 3-layer authorization (route, service, field) — see §4
- **Tests:** Integration tests verify cross-tenant access blocked, missing permission blocked
- **Audit:** Failed authorization attempts logged at WARN

11.2 A02: Cryptographic Failures

- **Mitigation:** TLS 1.3, AES-256 at rest, Argon2id passwords, no MD5/SHA-1
- **Tests:** SAST scans for weak crypto; CI fails on use

- **Audit:** Annual crypto review

11.3 A03: Injection

- **Mitigation:** Prisma parameterized queries everywhere; no `$queryRawUnsafe`
- **Tests:** SAST scans for raw SQL; integration tests for injection patterns
- **Audit:** Code review checklist item

11.4 A04: Insecure Design

- **Mitigation:** Threat modeling per feature; security design review for new modules
- **Tests:** Architecture review board for new modules
- **Audit:** Quarterly architecture review

11.5 A05: Security Misconfiguration

- **Mitigation:** Infrastructure as Code (Terraform); config validation at boot (zod); no default credentials
- **Tests:** CIS benchmark scanning; automated config drift detection
- **Audit:** Monthly config audit

11.6 A06: Vulnerable & Outdated Components

- **Mitigation:** `npm audit` on every PR; Dependabot for auto-PRs; Snyk for deep scanning
- **Tests:** CI fails on high/critical vulnerabilities
- **Audit:** Weekly dependency review

11.7 A07: Identification & Authentication Failures

- **Mitigation:** Argon2id, MFA, account lockout, session rotation
- **Tests:** Penetration tests for auth flows
- **Audit:** Annual pen test

11.8 A08: Software & Data Integrity Failures

- **Mitigation:** Signed commits; signed container images; SBOM (Software Bill of Materials); subresource integrity for frontend assets
- **Tests:** CI verifies signatures before deploy
- **Audit:** Continuous SBOM monitoring

11.9 A09: Security Logging & Monitoring Failures

- **Mitigation:** Audit framework (see Phase 0 §11); Sentry for errors; Loki for logs; Prometheus for metrics
- **Tests:** Audit log integrity tests
- **Audit:** Log review automation alerts on anomalies

11.10 A10: Server-Side Request Forgery (SSRF)

- **Mitigation:** No user-controlled URLs in outbound HTTP calls; URL whitelist for outbound integrations
- **Tests:** Integration tests with SSRF payloads
- **Audit:** Code review checklist item

---

12. SQL Injection Prevention

12.1 Parameterized Queries

- All DB access via Prisma — parameterized by default
- Raw queries use `Prisma.sql` template literal (parameterized):

```
// ✅ SAFE
const result = await prisma.$queryRaw`SELECT * FROM users
        WHERE email = ${email}`

// ❌ FORBIDDEN — string interpolation
const result = await prisma.$queryRawUnsafe(`SELECT * FROM
        users WHERE email = '${email}'`)
```

12.2 CI Enforcement

- ESLint rule `@suop/eslint-config/no-queryRawUnsafe` blocks `$queryRawUnsafe`
- Code review checklist item
- SAST scan (SonarQube) flags raw SQL

12.3 Dynamic Identifier Escaping

For dynamic table/column names (rare — usually a code smell): - Use Prisma's `Prisma.raw` with explicit allowlist - Never interpolate user input into identifiers - Example:

```
const allowedColumns = ['name', 'sku', 'status']
if (!allowedColumns.includes(sortColumn)) throw new
        ValidationError('Invalid sort column')
const result = await prisma.$queryRaw`SELECT $
        {Prisma.raw(sortColumn)} FROM products`
```

---

13. XSS (Cross-Site Scripting)

13.1 Output Encoding

- React auto-encodes all interpolated values (`{userInput}`) — no raw HTML by default
- `dangerouslySetInnerHTML` forbidden; if needed (rare), sanitize with `DOMPurify` first
- CI scans for `dangerouslySetInnerHTML` usage; requires code review approval

13.2 Content Security Policy

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'wasm-unsafe-eval';
```

```
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;  # allow images from S3
  font-src 'self' data:;
  connect-src 'self' https://api.sudhamrit.com;
  frame-ancestors 'none';
  form-action 'self';
  base-uri 'self';
  object-src 'none';
  upgrade-insecure-requests;
```

- No `unsafe-inline` for scripts (strict)
- `unsafe-inline` for styles (Tailwind requires; acceptable risk)
- No `unsafe-eval` for scripts (no `eval()` anywhere)

13.3 Other Security Headers

```
 X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Resource-Policy: same-origin
```

13.4 Cookie Security

- All cookies: `Secure; HttpOnly; SameSite=Strict` (or `Lax` for navigation cookies)
- No sensitive data in cookies other than refresh token
- Cookie prefix: `__Host-` for refresh token cookie (extra protection)

---

14. CSRF (Cross-Site Request Forgery)

14.1 Protection Strategy

- **API:** CSRF not vulnerable — API uses `Authorization: Bearer` header (not cookies), so CSRF cannot forge requests
- **Refresh token endpoint:** Uses cookie, but `SameSite=Strict` prevents CSRF
- **Server-side form submissions:** None (SPA + API architecture)

14.2 Double-Submit Cookie (Future, if Needed)

If server-side rendering with cookies is added: - Issue CSRF token in cookie + require in header for mutating requests - Verify match on server

---

15. SSRF (Server-Side Request Forgery)

15.1 Outbound HTTP Calls

- All outbound URLs validated against allowlist of domains
- Example: email service = `smtp.sendgrid.net`, file storage = `s3.amazonaws.com`
- User input never used as URL for outbound requests

15.2 URL Validation

```
const ALLOWED_OUTBOUND_DOMAINS = [
  'api.sendgrid.com',
  's3.ap-south-1.amazonaws.com',
  's3.amazonaws.com',
  'minio.local',
]

function validateOutboundUrl(url: string): void {
  const parsed = new URL(url)
```

```
  if (!ALLOWED_OUTBOUND_DOMAINS.includes(parsed.hostname)) {
    throw new SecurityError(`Outbound requests to ${parsed.hostname} not allowed`)
  }
  // Block internal IPs (prevent SSRF to metadata service)
  if (isInternalIp(parsed.hostname)) {
    throw new SecurityError(`Outbound requests to internal IPs forbidden`)
  }
}
```

15.3 Internal IP Blocking

- Block outbound requests to `127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `169.254.0.0/16` (AWS metadata service)
- Prevents SSRF attack from compromising cloud credentials

---

16. Audit Security

16.1 Audit Log Integrity

- Audit log table is **append-only** (no UPDATE, no DELETE) — enforced by PostgreSQL RLS
- Daily hash chain: each day's audit logs hashed; hash published to external system (S3 with object lock)
- Tampering detected by re-computing hash chain

16.2 Audit Log Access

- `audit:read` permission required for normal audit logs
- `audit:read:critical` required for high-severity logs (security events, permission changes)
- All audit log reads are themselves audit-logged (meta-audit)
- Audit logs never exported without approval

16.3 Audit Log Retention

- Hot: 1 year (online queryable)
- Warm: 1-3 years (archived, restorable)
- Cold: 3-7+ years (Glacier, compliance)
- Never deleted without legal sign-off

---

17. API Security

(see API Standards §18 for full details)

17.1 Key Controls

- All endpoints require authentication (except login, register, health, reference data)
- All endpoints rate-limited
- All inputs validated by zod schema
- All mutations require idempotency key
- All mutations audit-logged
- Sensitive endpoints (delete, blacklist, recall) require MFA re-authentication

17.2 Sensitive Endpoint Re-Authentication

For especially destructive actions: - `DELETE /api/v1/products/{id}` (hard delete — admin only) - `POST /api/v1/suppliers/{id}/blacklist` - `POST /api/v1/recalls` (initiate recall) - `POST /api/v1/_internal/users/{id}/revoke-sessions` - `POST /api/v1/_internal/tenants/{id}/purge` (purge tenant data)

Server requires: 1. Recent authentication (within last 5 minutes) 2. MFA code re-entered 3. Audit log at `CRITICAL` severity 4. Optional: 2-person rule (another admin approves)

---

18. Infrastructure Security

18.1 Network

- **VPC:** All resources in private VPC; only load balancer has public IP
- **Subnets:** App in private subnet; DB in isolated subnet (no internet egress)
- **Security groups:** Least privilege — app can reach DB only on port 5432; DB cannot reach anything
- **NAT gateway:** App egress to internet via NAT (for outbound integrations)
- **VPC endpoints:** S3, Secrets Manager via VPC endpoint (no public internet)

18.2 Database

- **No public IP** — only accessible from app subnet
- **TLS required** for all connections
- **Row-Level Security** for tenant isolation
- **Application role** has limited privileges (no DDL)
- **Migration role** separate, used only by CI/CD

18.3 Redis

- **Auth required** (`requirepass` or ACL)
- **TLS enabled**
- **No public IP** — only accessible from app subnet
- **Maxmemory policy:** `allkeys-lru` (cache mode; if lost, app rebuilds)

18.4 File Storage (S3 / MinIO)

- **Bucket private** — no public access
- **Access via signed URLs only** (15-min TTL)
- **Bucket policy** denies non-TLS requests
- **Versioning enabled** (recoverable from accidental delete)
- **Cross-region replication** for DR

18.5 Container Security

- **Non-root user** in container (`USER 1001`)
- **Read-only filesystem** (except `/tmp`)
- **No new privileges** flag (`--security-opt=no-new-privileges`)
- **Resource limits** (CPU, memory)
- **Image scanning** via Trivy before deploy
- **Base image:** `node:20-alpine` (small attack surface)

18.6 Kubernetes (Production)

If deploying via K8s: - **Network policies** restrict pod-to-pod communication - **Pod security standards** (restricted) - **RBAC** for K8s API - **Admission controllers** (OPA Gatekeeper) enforce policies - **Secrets** via external-secrets operator (sync from secrets manager)

---

19. Developer Security

19.1 Developer Machines

- **Disk encryption** mandatory (FileVault for Mac, BitLocker for Windows)
- **Screen lock** after 5 min idle
- **Antivirus** (corporate standard)
- **No production data on dev machines** — use sanitized staging data

19.2 Source Code Security

- **Signed commits** required (GPG or SSH signing)
- **Branch protection** on `main` and `develop`
- **PR review** required (1 reviewer for code, 2 for security-sensitive changes)
- **Secret scanning** via GitHub Advanced Security (or git-secrets locally)
- **Dependency scanning** via Dependabot

19.3 Access Management

- **Production access:** MFA required; JIT (just-in-time) access via approval workflow
- **Staging access:** All developers
- **DB access:** Via bastion host + SSH key only; no direct DB access from dev machines
- **Cloud console:** SSO + MFA; role-based access; audit logged

---

20. Incident Response

20.1 Incident Severity

Severity	Definition	Response Time
SEV-1	Active breach, data loss, ransomware	15 min
SEV-2	Vulnerability being exploited, limited impact	1 hour
SEV-3	Vulnerability discovered, not exploited	4 hours
SEV-4	Hardening opportunity, no active threat	1 week

20.2 Incident Response Plan

1. **Detect** (monitoring alert, user report, security researcher)
2. **Triage** (on-call engineer assesses severity)
3. **Contain** (revoke tokens, block IPs, isolate systems)
4. **Investigate** (logs, audit trail, forensics)
5. **Eradicate** (apply fix, rotate credentials)
6. **Recover** (restore service, verify integrity)
7. **Post-mortem** (within 7 days; root cause, action items)

20.3 Communication

- **Internal:** Slack incident channel; status page for employees
- **Customer:** Email notification within 72 hours of confirmed breach (GDPR / DPDP requirement)
- **Regulatory:** Notify FSSAI / CERT-In within 72 hours for applicable incidents
- **Public:** Blog post after containment (transparency)

---

21. Compliance

21.1 Frameworks

Framework	Applicability	Status
OWASP Top 10	All	Compliant (this document)
ISO 27001	Future (after v1.0)	Architecture ready
SOC 2 Type II	Future (after v1.0)	Architecture ready
FSSAI Cybersecurity	Food business	Compliant
DPDP Act 2023 (India)	Personal data	Compliant
GST	Invoice data	Compliant (retention)

21.2 Data Subject Access Requests (DPDP)

- User can request all their data: `GET /api/v1/_internal/users/me/data-export`
- User can request deletion: `DELETE /api/v1/_internal/users/me` (soft delete + audit; data retained per compliance)
- Response within 30 days

---

22. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-S1	Password policy	12+ chars, 3 of 4 categories	16+ chars (more strict)
Q-S2	MFA requirement	Admin/finance/ recall roles	All users
Q-S3	JWT algorithm	HS256 now → RS256 later	RS256 from start
Q-S4	Column encryption	pgcrypto for PII	Application-level only
Q-S5	Secrets manager	AWS Secrets Manager	HashiCorp Vault
Q-S6	Pen test frequency	Annual + after major changes	Quarterly
Q-S7	CSP strictness	Strict (no unsafe-inline scripts)	Permissive
Q-S8	Sensitive endpoint re-auth	MFA within 5 min	Password re-entry
Q-S9	Incident response	72-hour notification	24-hour (GDPR-style)
Q-S10	Production access	JIT (just-in-time)	Standing access

Approval Block

Approved by: _____ Date: _____

End of Enterprise Security Architecture Document